equip

Results for Python Developer

Saranya S Uncategorized

saranyaselvaraj4324@gmail.com

Rank: 9/12

1 Jul, 6:54 PM	1 hr 16 mins	13.46%	97%
Started At	Time Taken	Overall Score	Trust Score

Summary of Tests

Test Title	Type	Test Score	Trust Score	Finished
Quiz	Quiz	40.38%	100%	Yes
Compare and merge strings	Programming	-	97%	Yes
Sum Of All Credit Transactions	SQL	-	100%	Yes

Quiz

Report URL: https://equip.co/recruiter/assessments/XHTYf/attempts/FPzBQFg/?test_attempt_label=CxDgTVbX

Started At	Time Taken	Test Score	Trust Score
1 Jul, 6:54 PM	00:26:40	40.38%	100%

Why Can't I See The Questions?

Overall Performance

Questions across all difficulty levels

B Basic Mathematics

64%

4.50/7 Points

with -0.5 point

5	0	5	4	1
Questions	Unattempted	Attempted	Correct	Wrong

D Data Structures and Algorithms

55.0%

6/11 Points

with -1 point

6	0	6	4	2
Questions	Unattempted	Attempted	Correct	Wrong

S SQL

32%

2.25/7 Points

with -0.75 point

5	1	4	2	2
Questions	Unattempted	Attempted	Correct	Wrong

Python

21%

3/14 Points

with -2 points

9	0	9	3	6
Questions	Unattempted	Attempted	Correct	Wrong

Compare and merge strings

Report URL: https://equip.co/recruiter/assessments/XHTYf/attempts/FPzBQFg/?test_attempt_label=OQ2XvjsJ

Started At	Time Taken	Test Score	Trust Score
1 Jul, 7:30 PM	00:28:15	0%	97%

Submitted Code (Python3)

```python
#Please do not write code to take input.
# Just write the logic in the given function


def  merge_strings (string1,string2):
    i,j = 0,0
    result = []
while i <len (string1)
and j <len (string2):
 if string1[i]<
string2[j]:
result.append(string2[i])
i+=1
elif string1[i]>
string2[j]:
result.append(string2[j])
j+=1
else:
    result.append(string1[i])
    i+=1
    result.extend(string1[i:])
    result.extend(string2[j:])
    return '' . join (result)
    s1 = input().strip()
```

Results

4	0
Total Testcases	Successful Testcases

Compilation Error

```
Traceback (most recent call last):
  File
"/usr/lib/python3.5/py_compile.py",
line 125, in compile
    _optimize=optimize)
  File "<frozen
importlib._bootstrap_external>",
line 735, in source_to_code
  File "<frozen
importlib._bootstrap>", line 222,
in _call_with_frames_removed
```

Problem Statement

Problem Statement

You are given two strings `string1` and `string2` made of uppercase letters. You must create a third string `string3` out of them. These are the rules:

- Step 1: Compare the first letter in both strings and pick the smaller of the two. (for eg, if the words are `ABC` and `DEF`, you will pick `A` from the first string)
- Step 2: If both the letters are the same, pick the letter from `string1`
- Step 3: The letter you pick is removed from the original string and gets added to `string3`

- Step 4: You again compare the first two letters in the two strings like in Step 1 above and follow Steps 2 and 3

You follow this process until no letters are left in `string1` and `string2`.

Input & Output

Input Format

- The first line contains a string
- The second line contains a string

Input Constraint

- Both input strings are in upper case
- Each string has at least one alphabet

Output Format

A single string

Examples

Example 1

Input

```
ACA
BCF
```

Output

```
ABCACF
```

Explanation

The table below shows how the process works. We extract characters from both strings step-by-step.

string1	string2	string3

ACA	BCF	
CA	BCF	A
CA	CF	AB
A	CF	ABC
	CF	ABCA
	F	ABCAC
		ABCACF

Example 2

Input

```
RAJ
RAVI
```

Output

```
RAJRAVI
```

Explanation

The first letters to choose from are R and R since they are on the top of the stacks. R is chosen from the first string. Then, the options are A and R. A is chosen. Then the two stacks have J and R, so J is chosen. The current string is RAJ. As all the letters from the first stack have been used up, all the characters from the second stack can be appended to the resultant string.

Example 3

Input

```
DAVID
ROSE
```

Output

```
DAROSEVID
```

Explanation

The first letters to choose from are D and R. D is chosen from the first string. Then, the options are A and R. A is chosen. Then the two stacks have V and R, so R is chosen. The current string is DAR. Next, the letters to choose from are V and O. O is selected. This process continues until all the characters of both stack have been used up.

User Task

Your task is to complete the function `solution()` which takes 2 strings `string1` and `string2` and returns the expected answer. You don't need to read the input as a string, just write code within the function block directly.

Sum Of All Credit Transactions

Report URL: https://equip.co/recruiter/assessments/XHTYf/attempts/FPzBQFg/?test_attempt_label=6UzjHPG8

Started At	Time Taken	Test Score	Trust Score
1 Jul, 8:06 PM	00:04:41	0%	100%

Submitted Code (SQLite)

```sql
SELECT SUM(amount) AS total_credit
FROM transactions
WHERE transactions_type = 'credit'
AND DATE = '2017-05-27';

```

Results

2 Total Testcases

0 Successful Testcases

Testcases

Input	Expected Output	Output
INSERT INTO (1, "Rafael' (1, "Jane", (1, "Janice' (1, "Roger", INSERT INTO (2, "Montrea (3, "Sydney'	2500	Error: near l

Problem Statement

Database Schema

You have a database with tables

- `city`
- `accounts`
- `transactions`

The `city` table has the following columns:

- integer column `city_id` (primary key)
- text column `name`

The `accounts` table has the following columns:

- integer column `account_id` (primary key)
- text column `account_name`
- text column `account_type`
- integer column `city_id` references the `city_id` column of the `city` table (foreign key)

The `transactions` table has the following columns:

- integer column `transaction_id` (primary key)
- integer column `account_id` references the `account_id` column of the `accounts` table (foreign key)
- text column `transaction_type`
- text column `account_type`
- integer column `amount`
- date column `date`

User Task

Your task is to return the sum of all credit transactions on `27th May 2017`

Testcases

Sample Testcase 1

Table data

`city`

city_id	name
1	New York
2	Paris
3	Berlin

`accounts`

account_id	account_name	account_type	city_id
1	Jack	Savings	1

2	Alice	Current	2
3	Sweeney	Savings	1
4	David	Savings	3
5	Roger	Current	1

`transactions`

transaction_id	account_id	transaction_type	amount	date
1	1	Credit	1000	2017-05-27
2	1	Credit	200	2017-05-27
3	3	Credit	1500	2017-05-27
4	2	Savings	332	2017-02-23
5	3	Credit	1700	2017-05-21

Output

```
2700
```

Sample Testcase 2

Table data

`city`

city_id	name
1	Tokyo
2	New York
3	Barcelona

`accounts`

account_id	account_name	account_type	city_id
1	Nathan	Savings	1
2	Mira	Current	2
3	Katie	Savings	1
4	David	Savings	3
5	Roger	Current	1

5	Roger	Current	1

`transactions`

transaction_id	account_id	transaction_type	amount	date
1	1	Credit	1000	2017-05-27
2	1	Credit	200	2017-05-22
3	3	Credit	1500	2017-05-23
4	2	Savings	332	2017-02-23
5	3	Credit	1700	2017-05-21

Output

```
1000
```